SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section **Weboosta** is introducing the strategic solution of the identified problem statement highlighted in the Product Requirement Documentation. As the hospitality industry transitions from human-centric search to agent-led selection, this platform serves as the critical bridge, transforming unstructured hotel data into machine-readable signals.

1.1. Purpose:

The system analysis document highlights the technical scope and agentic-design decisions behind the ***Weboosta*** *development. This analysis ensures the platform transitions from simple LLM-chaining to a robust, tool-augmented diagnostic engine.*

So, the key element of system that has been covered are as followed:

1. ***Architecture****:*
    a. *Orchestration Layer: The system uses Gemini 3.1 Pro as the central "brain" to manage stateful reasoning across a multi-agent pipeline.*
    b. *Modular Services: Each agent—Discovery, Scraper, Analyzer, Optimizer, and Validator—functions as an independent logical unit, ensuring modularity and easier debugging.*
    c. *External Tool Integration: The architecture is tool-augmented, utilizing the Tavily Search API for competitive intelligence and Firecrawl/Playwright for high-fidelity web scraping.*
    d. *Cloud-Native Persistence: A Supabase/PostgreSQL backend handles the storage of optimization history, allowing for longitudinal benchmarking and session retrieval.*
2. ***Data Flows:***
    a. *Market Discovery & Competitive Intelligence: The pipeline initiates by validating natural language queries and identifying candidate properties through AI-optimized search via the Tavily API to establish official digital footprints.*
    b. *Deep-Web Research & Extraction: This node executes a high-fidelity crawl of the target property's entire web ecosystem using Firecrawl to ingest deep, unstructured data for signal identification.*
    c. *Multi-Modal Data Aggregation: Fragmented technical signals are synthesized into a cohesive, high-density property profile utilizing Gemini 3.1 Pro's large-context processing capabilities.*
    d. *AI Agent Decision Simulation: The system simulates a real-world autonomous travel agent evaluating the property against specific intent to calculate a baseline "Selection Score" and visibility rank.*
    e. *Visibility Gap Analysis: This diagnostic stage identifies precise technical friction points—such as absent trust signals or weak semantic relevance—that lead to property omission by AI engines.*

© UMHackathon 2026

f. *Autonomous AEO Signal Optimization: The engine automatically generates machine-legible Schema.org markup and entity-dense USPs designed to address the identified logical and structural gaps.*
g. *Programmatic QA Validation: A specialized Validator Agent performs a recursive audit of the optimized profile against original raw logs to ensure 100% factual grounding and eliminate hallucinations.*
h. *Recursive AI Re-simulation: The optimized data is re-evaluated through the simulation engine to mathematically quantify the improvement delta and project a measurable ROI for the property owner.*
i. *Human-in-the-Loop (HITL) Persistence: Final results are presented via a comparative dashboard for manual approval before the session state is committed to the Supabase cloud persistence layer.*

3. ***Model Process:***
The model illustrates the end-to-end workflow of the core diagnostic and optimization features: initiating a real-time competitive simulation to identify visibility gaps, executing a high-fidelity scrape of unstructured property data for logical gap analysis, generating machine-readable Schema.org and entity-dense metadata, and performing a recursive validation check to quantify the improvement delta before persisting the final session records.
4. ***Role of Reference:***
This document serves as the authoritative technical blueprint for the development, testing, and quality assurance teams to ensure the structural integrity of the Weboosta AEO Optimizer. It provides a comprehensive reference for backend and frontend engineering, guiding the implementation of the six-agent orchestration pipeline and its integration with external tool-augmentation. By establishing a shared logic for the system's stateful reasoning and persistence layers, this reference enables the team to maintain technical alignment and high-fidelity execution throughout the UMHackathon 2026 development cycle.

1.2. ***Background****:*

The hospitality industry currently operates on a human-centric marketing paradigm, prioritizing legacy SEO, review sentiment, and persuasive copy designed for travelers. However, as AI agents increasingly act as autonomous proxies for users in complex trip planning and property selection, a fundamental "visibility gap" has emerged. Most hotel listings remain optimized exclusively for human consumption, making their digital presence essentially "illegible" to the generative engines that now control the discovery layer. Consequently, high-quality properties face systemic omission from AI recommendations simply because their data is not structured for machine interpretability.

1. ***Previous Version:***
Traditional digital presence strategies for hotels are largely limited to manual content management and basic keyword optimization targeted at human search behavior. Earlier technical implementations in this space often relied on simple, one-shot LLM prompts or

© UMHackathon 2026

linear chaining, which lacked the ability to verify findings against real-time market data or competitive benchmarks. These legacy approaches provided no quantifiable method for measuring how a property is "perceived" by AI decision engines, nor did they generate the structured JSON-LD and Schema.org metadata required for autonomous agentic extraction.

2. ***Changes in Major Architectural Components:***
   The architecture undergoes a transformative shift from a static, linear processing model to a stateful, multi-agent orchestration pipeline. This transition is headlined by the deployment of the Gemini LLM (Gemini 3.1 Pro) as a central reasoning engine, which manages the state and logic flow between independent micro-agent services including Discovery, Scraper, Analyzer, Optimizer, and Validator. This modular approach allows the system to execute a complex "Discover Why?" diagnostic workflow, where each stage of the pipeline functions as an isolated logical unit, significantly enhancing system reliability and allowing for targeted debugging of individual agent outputs.

   Furthermore, the integration of real-time tool-augmentation serves as a major architectural pillar, grounding the AI's reasoning in live web data via the Tavily Search API and Firecrawl. This replaces traditional static knowledge with dynamic competitive intelligence, ensuring that visibility scores are based on current market conditions. The addition of a Cloud-Native Persistence layer using Supabase and PostgreSQL introduces a long-term data strategy, enabling the system to store session history and track improvement deltas over time. These backend enhancements are surfaced through a sophisticated frontend architecture utilizing Three.js and React Three Fiber, which provides an interactive 3D visualization of the optimization process, effectively bridging the gap between technical AI scoring and user-centric insight.

3. ***New capabilities that are only introduced in this:***
   The current version of the platform introduces a stateful agentic orchestration layer built on LangGraph, which allows the system to manage complex, non-linear transitions between specialized agents including Discovery, Web Research, Gap Analysis, Optimization, and Validation. This architectural shift enables high-fidelity signal extraction through the integration of Firecrawl, allowing the Web Research agent to ingest deep, unstructured data from multiple hotel sub-pages to identify nuanced visibility signals that traditional crawlers often overlook. Furthermore, the system performs entity-dense AI Engine Optimization (AEO) synthesis by transforming property descriptions into machine-readable Schema.org and JSON-LD markup specifically designed for extraction by autonomous agents. This capability is paired with persona-based visibility simulation, where the AI Simulator and Resimulator nodes test property performance against Budget, Luxury, Family, and Business traveler personas to predict how different algorithmic weights impact a hotel's ranking. To maintain transparency, the platform includes interactive 3D spatial visualization using React Three

© UMHackathon 2026

Fiber and Three.js to map the internal state of the agentic pipeline to a real-time 3D cube interface. Finally, an automated validation and confidence scoring layer ensures the factual integrity of all optimizations by cross-referencing generated content against original raw logs before the data is committed to the Supabase persistence layer.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Hospitality Executives & Property Owners	*Strategic oversight and budget allocation for digital transformation initiatives.*	*A measurable competitive advantage and increased organic bookings through a quantifiable "Visibility Score Delta."*
Revenue & Digital Marketing Managers	*Managing the property's digital assets, brand voice, and final approval of AI-generated signal optimizations.*	*Intuitive dashboards that automate technical Schema.org generation and reduce reliance on high-commission Online Travel Agencies (OTAs).*
AI Decision Engines (Gemini, OpenAI, etc.)	*Acting as the "Digital Gatekeepers" that index property data to provide autonomous traveler recommendations.*	*High-fidelity, machine-readable JSON-LD data that adheres to strict entity-driven standards for seamless ingestion.*
Product & Engineering Team (Internal)	*Maintaining the structural integrity of the LangGraph orchestration and the factual grounding of the multi-agent pipeline.*	*A reliable, scalable backend architecture that delivers non-hallucinated, production-ready technical outputs with minimal latency.*
Strategic Partners (API Providers)	*Providing the real-time data retrieval (Tavily/Firecrawl) and LLM reasoning (Gemini) required for the diagnostic workflow.*	*Stable API availability and high-accuracy data extraction to support the system's stateful "Discover Why?" logic.*
Traveler Personas (End Consumers)	*Providing the natural language queries that drive the AI recommendation logic.*	*Accurate, relevant property suggestions that perfectly match their specific needs (e.g., Budget, Luxury, or Family-oriented) based on optimized signals.*
Business C-suite	*Strategic alignment and measuring platform ROI*	*Quantifiable increase in market visibility and clear return on investment.*

© UMHackathon 2026

Developers (Client-side)	*Integrating the machine-ready outputs into property websites*	*Validated, easy-to-implement JSON-LD and Schema.org metadata.*
AEO Specialists	*Leveraging the tool's diagnostics to optimize content for AI decision engines*	*Detailed visibility gap analysis and persona-based performance simulations.*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based AI Engine Optimization (AEO) Diagnostic & Optimization Dashboard*
Architecture	*Stateful Multi-Agent Orchestration (LangGraph) with Cloud-Native Persistence*

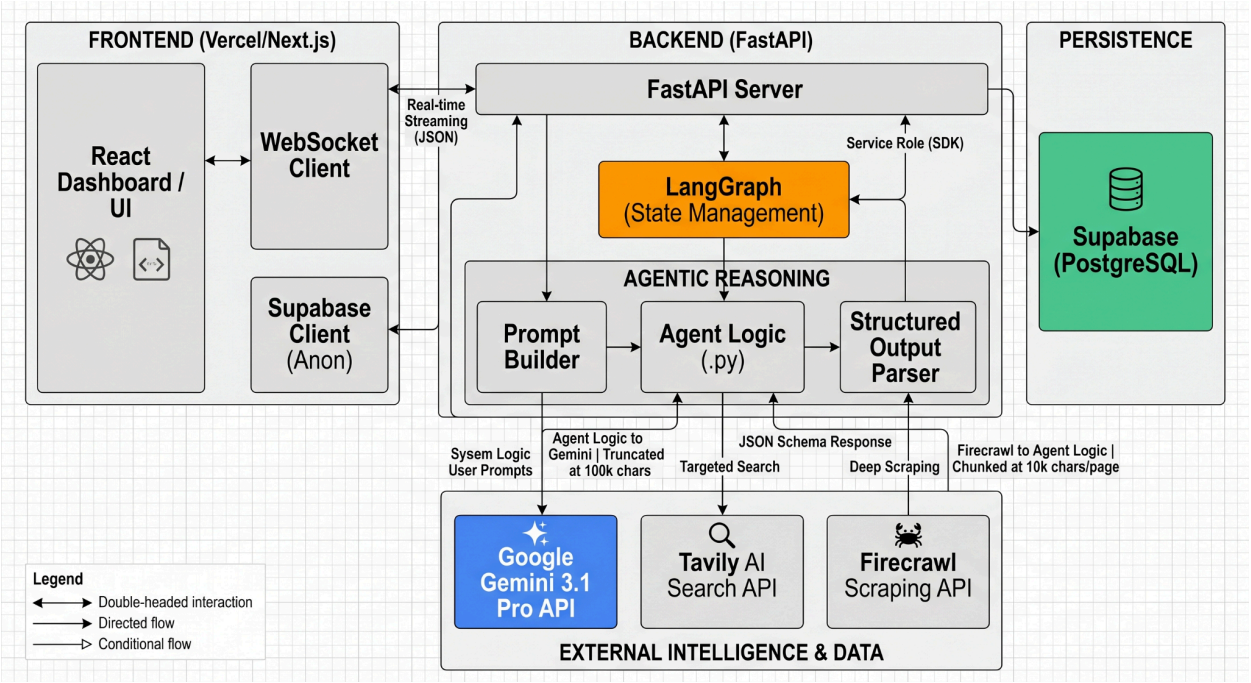
Weboosta (UMami) is primarily structured as a client-server-based system featuring a centralized Web-based Property Optimization Dashboard as the primary client. This interface communicates with a robust FastAPI Backend API layer, which orchestrates the system's core diagnostic and optimization logic. The backend is organized into several independent agentic micro-services: the Web Research & Scraper Service, the Gap Analysis Service, the AEO Optimization Engine, and the Simulation & Validation Service. All these services are statefully coordinated by a LangGraph Orchestrator and share a cloud-native PostgreSQL database hosted on Supabase, which ensures persistent storage for property metadata, competitive benchmarks, and optimization histories.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how Gemini is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

© UMHackathon 2026

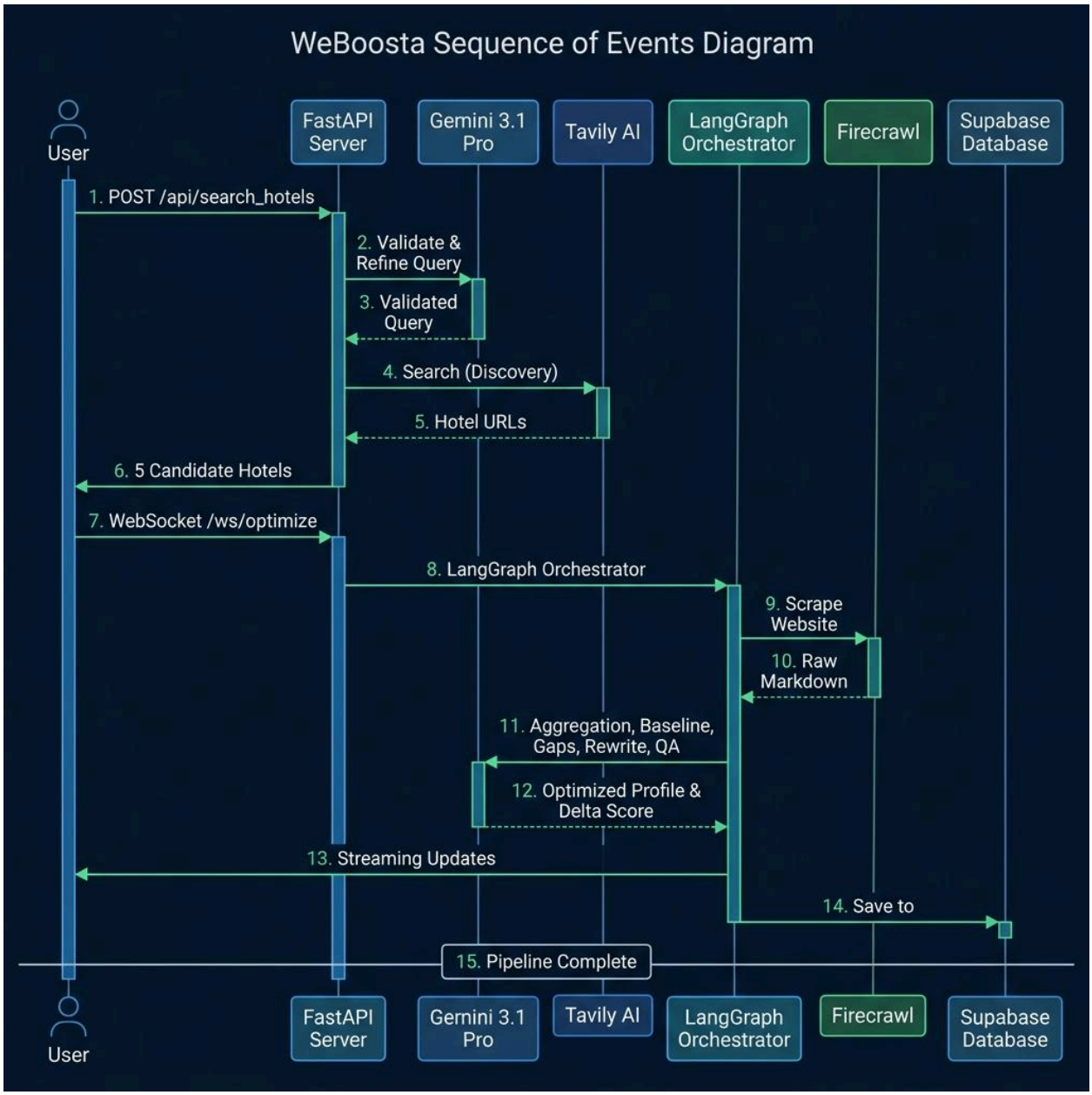
2.1.3 Dependency Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram



© UMHackathon 2026

2.2. Technological Stack

2.2.1. ***Frontend:*** *Next.js 15, React 19, Tailwind CSS v4, Three.js, React Three Fiber, Framer Motion*

2.2.2. ***Backend:*** *FastAPI, LangGraph, LangChain, Gemini 3.1 Pro, Tavily Search API, Firecrawl, WebSockets*

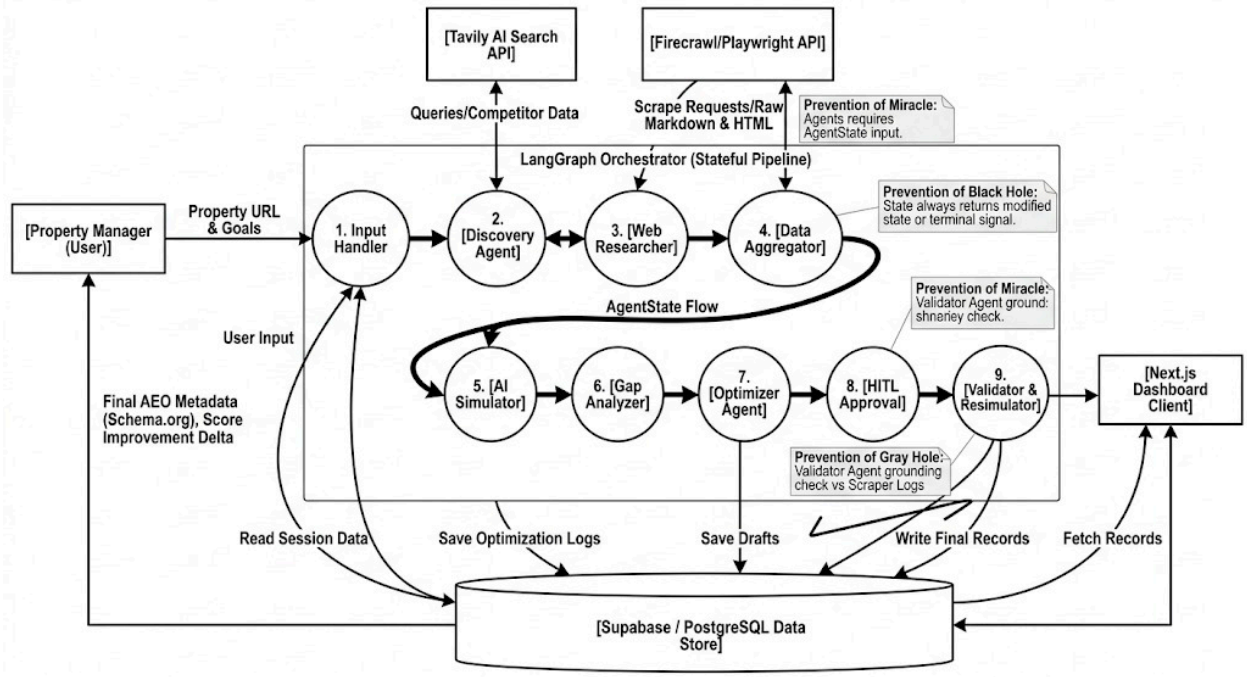
2.2.3. ***Database:*** *Supabase, PostgreSQL*

2.2.4. ***Cloud/Deployment:*** *Vercel, Railway, Supabase Cloud*

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

HOTEL (hotel_url, hotel_name)

OPTIMIZATION_RECORD (id, created_at, query, hotel_url, baseline_score, optimized_score, reasoning, original_profile, optimized_profile)

RECORD_SOURCE (id, source)

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.* ***[e.g. in Table]***

#	Features	Description
1	**"Discover Why?" Multi-Agent Pipeline**	*Orchestrates a **stateful 9-node workflow** powered by **LangGraph** to perform recursive diagnostics on property visibility within autonomous AI search ecosystems.*
2	**Deep-Web Signal Extraction**	*Leverages **Firecrawl and Playwright** to ingest and structure complex, multi-page property data, identifying nuanced signals that traditional SEO crawlers typically overlook.*
3	**Persona-Based Visibility Simulation**	*Evaluates property performance against four traveler archetypes—**Budget, Luxury, Family, and Business**—to calculate baseline visibility scores and identify specific audience-based ranking gaps.*
4	**Autonomous AEO Signal Generation**	*Automatically transforms identified data gaps into **machine-legible Schema.org (JSON-LD)** and entity-dense metadata specifically designed for indexing by "Answer Engines."*
5	**Interactive 3D Agentic Visualization**	*Utilizes **React Three Fiber** and **Three.js** to provide a real-time, interactive 3D spatial representation of the*

© UMHackathon 2026

		internal AI reasoning states and optimization progress.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	***Requirements***	***Implementation***
Scalability	*The system must support simultaneous multi-agent optimization sessions without degrading the performance of the* ***LangGraph state machine****.*	*Utilization of* ***FastAPI's asynchronous execution*** *and* ***Supabase connection pooling*** *to handle high-concurrency database transactions across distributed agent nodes.*
Reliability & Accuracy	*Generated* ***AEO metadata and Schema.org markup*** *must achieve a 100% factual grounding rate to prevent property delisting or AI engine hallucinations.*	*A mandatory* ***Validator Agent*** *performs a programmatic audit by cross-referencing* ***Optimizer outputs*** *against the* ***Raw Scraper Logs*** *before any data is committed to the persistence layer.*
Maintainability	*The system must allow for the independent refinement of specialized agents (e.g., updating the Scraper logic) without requiring a full pipeline rewrite.*	*Implementation of a* ***Modular Agentic Architecture*** *where each node (Discovery, Researcher, Optimizer) is isolated in the* ***LangGraph workflow****, communicating solely through a shared* ***AgentState*** *object.*

© UMHackathon 2026

Performance (Latency)	*The user interface must provide immediate visual feedback during long-running LLM reasoning tasks to maintain user engagement (p95 < 4s for state updates).*	*Use of **Asynchronous background tasks** in the backend and a real-time **3D React Three Fiber (R3F) visualization** that reflects the current pipeline state through interactive UI transitions.*
Cost Efficiency	*The average operational cost per user optimization session must be maintained below a budget threshold of **$0.15 USD**.*	*Strategic deployment of **Recursive Summarization** for oversized web inputs and a **24-hour result caching** mechanism in the **Supabase** database to prevent redundant LLM calls for repeated property URLs.*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a. ***Direct OTA Synchronisation:*** *Automated API integration with **Online Travel Agencies (OTAs)** like Booking.com, Expedia, and Airbnb to push optimized metadata and amenities directly to property listings without manual copy-pasting.*

b. ***Specialized F&B Optimization Vertical:*** *Adaptation of the core AEO logic to support **Food & Beverage (F&B)** entities. This includes generating specialized Schema.org (e.g.,* `Restaurant`, `Menu`*) to index menu items, dietary restrictions, ingredients, and real-time reservation availability, enabling AI agents to handle complex diner queries.*

c. ***Multi-Framework Code Export:*** *Expansion of the code generation engine beyond basic **HTML/CSS** to support modern development frameworks such as **React, Swift (iOS), and Kotlin (Android)** for native application optimization.*

d. ***Real-Time Dynamic Pricing Alignment:*** *Integrating live pricing and inventory data into the **AI Simulation** nodes to determine how fluctuating rates impact a property's visibility score in real-time.*

© UMHackathon 2026

e. ***Multi-Modal Visual Signal Extraction:*** *Deployment of computer vision agents to analyze property photos and* ***360-degree virtual tours****, automatically extracting amenity signals (e.g., "sea view," "ergonomic workspace") directly from visual assets.*

f. ***Cross-Industry Vertical Expansion:*** *Adapting the* ***AI Engine Optimization*** *logic to serve small-to-medium enterprises (SMEs) in sectors beyond hospitality, including* ***Retail and Professional Services****.*

g. ***Autonomous Social Sentiment Integration:*** *Incorporating real-time sentiment analysis from social media platforms (Instagram, TikTok) to adjust visibility scores based on current viral trends and guest-generated content.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: To maintain technical stability and semantic accuracy, the team employs a controlled Grayscale Rollout and A/B testing strategy for major updates, such as the release of the Specialized F&B Optimization Vertical or new LLM reasoning models. Initially, new optimization logic is routed to only 5% of active properties, during which time system logs are rigorously monitored for regressions in technical metrics like Token Latency and qualitative metrics like the Validator Agent's factual grounding score; if no issues are detected, traffic is incrementally increased to 100%. Following this stabilization, A/B testing is conducted to measure business impact, splitting the user base into a control variant using existing strategies and a test variant using the new update, with the superior variant determined by which achieves a higher quantifiable Score Improvement Delta in simulated AI travel agent queries.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: To mitigate critical system anomalies—such as a spike in LangGraph pipeline failures, excessive token latency exceeding p95 thresholds, or a significant drop in the Validator Agent's factual grounding confidence during a rollout—Weboosta (UMami) utilizes an automated Emergency Rollback (ER) mechanism integrated into the CI/CD pipeline. When critical health thresholds are breached in production, the ER is immediately triggered, causing the continuous deployment infrastructure (managed via Vercel for the frontend and Render/Railway for the backend) to automatically redeploy the "Golden Release." This Golden Version represents the last certified, fully stable, and thoroughly validated snapshot of the application code and database schema within Supabase, ensuring immediate service restoration and the preservation of data integrity while the engineering team investigates the root cause of the anomaly.*

© UMHackathon 2026

4.1.3. Priority Matrix:

4.1.3.1. ***P1 Critical:*** *Trigger condition is the Validator Agent reporting a factual grounding score below 90% for over 15 minutes or a pipeline failure rate exceeding 10%, and the action is an automatic Emergency Rollback (ER) to the last stable Golden Release on Vercel and Railway.*

4.1.3.2. ***P High:*** *Trigger condition is a critical external dependency failure (Tavily AI, Firecrawl, or Gemini API) affecting more than 25% of sessions, and the action is to issue a high-priority alert to the Backend Lead, rotate API keys, and display a service notice on the dashboard.*

4.1.3.3. ***P3 Medium:*** *Trigger condition is token latency consistently exceeding the p95 threshold of 10 seconds per state transition, and the action is to log the issue in the backlog for technical debt review and analyze node-specific execution logs for prompt optimization.*

4.1.3.4. ***P4 Low:*** *Trigger condition is the presence of cosmetic UI inconsistencies or minor jitters in the 3D React Three Fiber visualization, and the action is to log the bug in the project backlog for resolution during standard maintenance cycles.*

4.2. ***Monitoring****:*

4.2.1. The monitoring architecture for ***Weboosta (UMami)*** *is built on a proactive, multi-layered approach that prioritizes infrastructure stability, agentic reasoning accuracy, and cost-efficient execution. At the base layer, the team implements* ***infrastructure monitoring*** *using native analytics from* ***Vercel and Railway*** *to ensure a* ***99.9% availability target****, while simultaneously tracking the health of critical external dependencies like* ***Tavily AI and Firecrawl****. Because the platform relies on a complex* ***stateful LangGraph pipeline****, the monitoring system goes beyond simple uptime checks to track* ***State Transition Health****, ensuring that the* ***AgentState*** *object successfully traverses all nine nodes—from* ***Market Discovery*** *to the final* ***Resimulator****—without encountering infinite loops or data "Black Holes."*

4.2.2. To detect damage to system health in real-time, the platform utilizes a specialized ***Validator Watchdog*** *that monitors the* ***Factual Grounding Score*** *of every AI-generated optimization. If this score, which measures how well the generated metadata aligns with raw scraper logs, falls below a* ***90% confidence threshold*** *for more than 15 minutes, it indicates widespread* ***LLM hallucinations*** *and triggers a* ***P1 Critical Alert****. This specific trigger automatically activates the* ***Emergency Rollback (ER)*** *mechanism, causing the CI/CD pipeline to redeploy the last certified* ***"Golden Release"*** *to protect user data integrity. Performance-based triggers are also set for* ***Token Latency****; should the backend API response time exceed a* ***p95 threshold of 10 seconds****, the system logs a* ***P3 incident****, alerting the engineering team to bottlenecks in the reasoning pipeline.*

4.2.3. Beyond automated rollbacks, the monitoring plan incorporates a tiered ***alerting and recovery protocol*** *managed through* ***Sentry and Supabase audit logs****. When operational*

© UMHackathon 2026

*impairments occur—such as consecutive **5xx errors** from external APIs—a **P-High trigger** dispatches immediate notifications to the **Backend Lead**, while the frontend dashboard is updated with a **service notice** to maintain transparency. This integrated monitoring loop ensures that every aspect of the ecosystem, from the low-level **PostgreSQL** transactions to the high-level **3D visualizations**, is under constant surveillance, allowing for rapid recovery and continuous delivery throughout the **UMHackathon 2026**.*

4.3. Assumptions: *The following are some key operational and environmental conditions that are assumed to be valid throughout both testing and in the deployed prototype:*

a. Stable internet access is maintained by the user to interact with the Next.js dashboard.
b. Continuous outbound connectivity exists for the backend to communicate with external API providers.
c. Reliable service uptime for third-party dependencies, specifically Gemini, Tavily AI, and Firecrawl.
d. Target property websites are accessible via standard protocols and do not utilize impenetrable anti-bot security that blocks Firecrawl.
e. Market intelligence benchmarks retrieved via the Tavily API are representative of current hospitality trends.
f. WebGL-enabled browsers (Chrome, Firefox, Edge) are used to support the Three.js and React Three Fiber (R3F) 3D visualizations.
g. Sufficient client-side GPU performance is available to render real-time agentic state transitions without significant latency.
h. Active Human-in-the-Loop (HITL) participation occurs at the Approval Node to move the AgentState to the final validation stage.
i. PostgreSQL Row-Level Security (RLS) effectively maintains data isolation between different property manager accounts within Supabase.
j. Valid URL inputs are provided by users that align with the platform's hospitality optimization logic.

4.4. **External Dependencies:** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	***Purpose***	***Risks***

© UMHackathon 2026

Gemini 3.1 Pro	Serves as the **core LLM reasoning engine** for the 9-stage LangGraph pipeline, handling multi-step reasoning, persona simulation, and structured AEO metadata generation.	**High Risk:** API rate limits or model hallucinations could compromise the factual integrity of optimizations. **Mitigation:** Implementation of the **Validator Agent** for factual grounding and a **Graceful Degradation** state for users.
Tavily AI Search API	Orchestrates **Market Discovery** by performing high-speed, AI-optimized searches to retrieve real-time competitive benchmarks and market intelligence.	**Medium Risk:** Downtime or quota exhaustion would halt the competitor analysis phase. **Mitigation:** Fallback search logic and **24-hour result caching** in the Supabase persistence layer.

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

Day	Main Focus	Justification
1 - 3	Project Planning	Come up a comprehensive plan backed by real data.
4 - 8	Prototyping	Design an initial prototype with detailed documentation
9 - 10	QA, Testing, Debugging	Provides ample time to QA in the case of errors, providing the development team time to debug and reiterate their code.

© UMHackathon 2026

5.2. *Team Members Role:*

5.2.1. Leong Wui Yip – Pitching Video (Director, Recording & Editing)
5.2.2. Francis Paul Narcis – Documentation (PRD & System Analysis)
5.2.3. Muhammad Ahmad – QA & Pitch Deck
5.2.4. Shariq Nauman – Backend Developer
5.2.5. Lee Ping Xian – UI/UX Designer & Workflow System Architect

5.3. *Recommendations:*

*5.3.1. Implement **Redis Cache** to store frequently accessed market intelligence from Tavily and Firecrawl to minimize redundant API calls. This significantly reduces latency for recurring property URLs processed within the **9-stage pipeline**.*

*5.3.2. Integrate a message broker like **RabbitMQ** to handle long-running agentic tasks asynchronously and prevent the **FastAPI backend** from blocking. This ensures the architecture remains responsive during high-concurrency reasoning loads.*

*5.3.3. Utilize **PostgreSQL Read Replicas** in Supabase to separate heavy analytical read queries from primary transactional state updates. This strategy optimizes the performance of the **Next.js Records Table** while maintaining the integrity of the LangGraph state.*

*5.3.4. Apply the **Circuit Breaker pattern** to external API calls to mitigate cascading failures if Gemini or Tavily services experience downtime. This allows the system to transition into a **graceful degradation state** by serving previously cached data.*

*5.3.5. Deploy an **Edge Content Delivery Network (CDN)** to cache static assets and 3D visualization components closer to the user's geographic location. This reduces the initial load time for the **React Three Fiber (R3F) environment** and improves overall performance.*

© UMHackathon 2026